



TCC805x MCU BSP

API Specification for Clock Control Unit

Rev. 1.01 [A]

2021-01-22

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit <http://www.telechips.com> for more information.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Source Files.....	4
3.1	Location of Source Files.....	4
3.2	Simple Description of Source Files.....	4
4	Enumerations	5
4.1	CLOCKPll_t.....	5
4.2	CLOCKMpll_t	6
4.3	CLOCKPclkctrlSel_t	7
4.4	CLOCKPeri_t.....	8
4.5	CLOCKIobus_t.....	9
5	Constants.....	11
5.1	Clock Rate	11
6	Structures	12
6.1	CLOCKPms_t.....	12
6.2	CLOCKClkCtrl_t.....	13
6.3	CLOCKPclkCtrl_t	14
7	APIs	15
7.1	CLOCK_Init	15
7.2	CLOCK_SetPllRate.....	16
7.3	CLOCK_GetPllRate	17
7.4	CLOCK_SetPllDiv.....	18
7.5	CLOCK_SetClkCtrlRate.....	19
7.6	CLOCK_GetClkCtrlRate	20
7.7	CLOCK_IsPeriEnabled.....	21
7.8	CLOCK_EnablePeri	22
7.9	CLOCK_DisablePeri	23
7.10	CLOCK_GetPeriRate	24
7.11	CLOCK_SetPeriRate.....	25
7.12	CLOCK_IsIobusPwdn.....	26
7.13	CLOCK_EnableIobus	27
7.14	CLOCK_SetIobusPwdn.....	28
7.15	CLOCK_SetSwReset	29
8	How to use CLOCK Driver	30
8.1	Initialize CLOCK Driver	30
8.2	Set Peripheral Clock	30
8.3	Set Peripheral Bus Clock.....	30
9	References	31
10	Revision History	32
	Rev. 1.01: 2021-01-22	32
	Rev. 1.00: 2020-11-12	32

Figures

Figure 3.1 Location of Source Files	4
---	---

Tables

Table 2.1 Terms and Abbreviation	3
--	---

1 INTRODUCTION

This document describes the clock control unit device driver (hereinafter referred to as CLOCK driver) of TCC805x MCU BSP. For more detailed information of the clock control unit in the TCC805x MCU Sub-system, refer to Chapter 21 Clock Control Unit (CCU) of the “*PART 11-MICOM SUB-SYSTEM*” in the “*TCC805x Full Specification*”. [1]

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

CCU	Clock Control Unit
PLL	Phase-Locked Loop
XIN	Clock Signal from PMU (Crystal Oscillator Clock)
XTIN	Clock Signal from RTC
VCO	Voltage-controlled oscillator
FIN	Input frequency
FOUT	Output frequency
PCLK	Peripheral Clock

3 SOURCE FILES

3.1 Location of Source Files

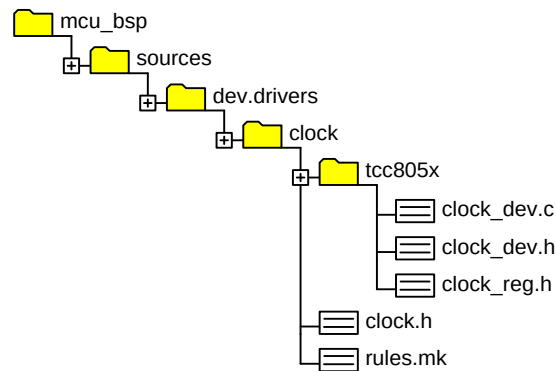


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- clock.h
 - External APIs header to set clock
- clock_reg.h
 - Defines of clock register
- clock_dev.h
 - Clock driver header
- clock_dev.c
 - Clock driver source
- rules.mk
 - rules used for build

4 ENUMERATIONS

4.1 CLOCKPLL_t

Enumeration variables for PLL channel index

Declaration

```
typedef enum CLOCK_PLL
{
    CLOCK_PLL_MICOM_0    = 0,
    CLOCK_PLL_MICOM_1    = 1
} CLOCKPLL_t;
```

Description

<i>Value</i>	<i>Description</i>
<i>CLOCK_PLL_MICOM_0</i>	<i>PLL0 selection index</i>
<i>CLOCK_PLL_MICOM_1</i>	<i>PLL1 selection index</i>

Remark

None

4.2 CLOCKMpll_t

Enumeration variables for MCU PLL source index

Declaration

```
typedef enum CLOCK_M_PLL
{
    CLOCK_MPLL_0      = 0,
    CLOCK_MPLL_1      = 1,
    CLOCK_MPLL_DIV_0   = 2,
    CLOCK_MPLL_DIV_1   = 3,
    CLOCK_MPLL_XIN     = 4
} CLOCKMpll_t;
```

Description

Value	Description
CLOCK_MPLL_0	PLL0 for FIN source clock
CLOCK_MPLL_1	PLL1 for FIN source clock
CLOCK_MPLL_DIV_0	Divided PLL0 for FIN source clock
CLOCK_MPLL_DIV_1	Divided PLL1 for FIN source clock
CLOCK_MPLL_XIN	XIN for FIN source clock

Remark

None

4.3 CLOCKPclkctrlSel_t

Enumeration variables for MCU Peripheral PLL source index

Declaration

```
typedef enum CLOCK_PCLK_CTRL_SEL
{
    CLOCK_PCLKCTRL_SEL_PLL0      = 0,
    CLOCK_PCLKCTRL_SEL_PLL1      = 1,
    CLOCK_PCLKCTRL_SEL_XIN       = 5,
    CLOCK_PCLKCTRL_SEL_PLL0DIV   = 10,
    CLOCK_PCLKCTRL_SEL_PLL1DIV   = 11,
    CLOCK_PCLKCTRL_SEL_XINDIV    = 23
} CLOCKPclkctrlSel_t;
```

Description

Value	Description
CLOCK_PCLKCTRL_SEL_PLL0	PLL0 output for peripheral source clock selection
CLOCK_PCLKCTRL_SEL_PLL1	PLL1 output for peripheral source clock selection
CLOCK_PCLKCTRL_SEL_XIN	XIN output for peripheral source clock selection
CLOCK_PCLKCTRL_SEL_PLL0DIV	Divided PLL0 output for peripheral source clock selection
CLOCK_PCLKCTRL_SEL_PLL1DIV	Divided PLL1 output for peripheral source clock selection
CLOCK_PCLKCTRL_SEL_XINDIV	Divided XIN output for peripheral source clock selection

Remark

None

4.4 CLOCKPeri_t

Enumeration variables for peripheral index

Declaration

```
typedef enum CLOCK_PERI
{
    CLOCK_PERI_SFMC      = 0,
    CLOCK_PERI_CAN0      = 1,
    CLOCK_PERI_CAN1      = 2,
    CLOCK_PERI_CAN2      = 3,
    CLOCK_PERI_GPSB0     = 4,
    CLOCK_PERI_GPSB1     = 5,
    CLOCK_PERI_GPSB2     = 6,
    CLOCK_PERI_GPSB3     = 7,
    CLOCK_PERI_GPSB4     = 8,
    CLOCK_PERI_GPSB5     = 9,
    CLOCK_PERI_UART0     = 10,
    CLOCK_PERI_UART1     = 11,
    CLOCK_PERI_UART2     = 12,
    CLOCK_PERI_UART3     = 13,
    CLOCK_PERI_UART4     = 14,
    CLOCK_PERI_UART5     = 15,
    CLOCK_PERI_I2C0      = 16,
    CLOCK_PERI_I2C1      = 17,
    CLOCK_PERI_I2C2      = 18,
    CLOCK_PERI_PWM0      = 19,
    CLOCK_PERI_PWM1      = 20,
    CLOCK_PERI_PWM2      = 21,
    CLOCK_PERI_ICTC0     = 22,
    CLOCK_PERI_ICTC1     = 23,
    CLOCK_PERI_ICTC2     = 24,
    CLOCK_PERI_ADC        = 25,
    CLOCK_PERI_TIMER0     = 26,
    CLOCK_PERI_TIMER1     = 27,
    CLOCK_PERI_TIMER2     = 28,
    CLOCK_PERI_TIMER3     = 29,
    CLOCK_PERI_TIMER4     = 30,
    CLOCK_PERI_TIMER5     = 31,
    CLOCK_PERI_MAX        = 32
} CLOCKPeri_t;
```

Description

Value	Description
CLOCK_PERI_xxxx	Peripheral clock control offset. Refer to Register Map (0x1b931000) in CCU in “TCC805x Full Specification”. [1]
CLOCK_PERI_MAX	Maximum Peripheral clock control offset

Remark

None

4.5 CLOCK_Iobus_t

Enumeration variables for peripheral index

Declaration

```
typedef enum CLOCK_IO_BUS
{
    CLOCK_IOBUS_NSAID_FILTER_BUS      = 0,
    CLOCK_IOBUS_NSAID_FILTER_CORE     = 1,
    CLOCK_IOBUS_NSAID_CFG_BUS         = 2,
    CLOCK_IOBUS_SGID_FILTER_BUS       = 3,
    CLOCK_IOBUS_SGID_FILTER_CORE      = 4,
    CLOCK_IOBUS_SGID_CFG_BUS          = 5,
    CLOCK_IOBUS_SFMC_BUS               = 6,
    CLOCK_IOBUS_SFMC_MID_FILTER_BUS   = 7,
    CLOCK_IOBUS_SFMC_MID_CFG_BUS      = 8,
    CLOCK_IOBUS_SFMC_BRIDGE           = 9,
    CLOCK_IOBUS_SFMC                   = 10,
    CLOCK_IOBUS_GIC                    = 15,
    CLOCK_IOBUS_GIC_BRIDGE            = 16,
    CLOCK_IOBUS_GPIO_BUS              = 17,
    CLOCK_IOBUS_SOC400                = 18,
    CLOCK_IOBUS_JTAG                  = 19,
    CLOCK_IOBUS_DMA_GRP               = 20,
    CLOCK_IOBUS_CAN_GRP               = 21,
    CLOCK_IOBUS_CAN0                  = 22,
    CLOCK_IOBUS_CAN1                  = 23,
    CLOCK_IOBUS_CAN2                  = 24,
    CLOCK_IOBUS_CAN_BUS               = 25,
    CLOCK_IOBUS_UART_GRP              = 32,
    CLOCK_IOBUS_UART0                 = 33,
    CLOCK_IOBUS_UART1                 = 34,
    CLOCK_IOBUS_UART2                 = 35,
    CLOCK_IOBUS_UART3                 = 36,
    CLOCK_IOBUS_UART4                 = 37,
    CLOCK_IOBUS_UART5                 = 38,
    CLOCK_IOBUS_UDMA0                 = 39,
    CLOCK_IOBUS_UDMA1                 = 40,
    CLOCK_IOBUS_UDMA2                 = 41,
    CLOCK_IOBUS_UDMA3                 = 42,
    CLOCK_IOBUS_UDMA4                 = 43,
    CLOCK_IOBUS_UDMA5                 = 44,
    CLOCK_IOBUS_UART_BUS              = 45,
    CLOCK_IOBUS_I2C_GRP               = 46,
    CLOCK_IOBUS_I2C0                  = 47,
    CLOCK_IOBUS_I2C1                  = 48,
    CLOCK_IOBUS_I2C2                  = 49,
    CLOCK_IOBUS_I2C_M_PORTCFG         = 50,
    CLOCK_IOBUS_PWM_GRP               = 51,
    CLOCK_IOBUS_PWM0                  = 52,
    CLOCK_IOBUS_PWM1                  = 53,
    CLOCK_IOBUS_PWM2                  = 54,
    CLOCK_IOBUS_PWM_BUS               = 55,
    CLOCK_IOBUS_ICTC_GRP              = 56,
    CLOCK_IOBUS_ICTC0                 = 57,
    CLOCK_IOBUS_ICTC1                 = 58,
```

```

CLOCK_IOBUS_ICTC2          = 59,
CLOCK_IOBUS_ADC             = 60,
CLOCK_IOBUS_MAILBOX        = 61,
CLOCK_IOBUS_TIMER_GRP      = 64,
CLOCK_IOBUS_TIMER0         = 65,
CLOCK_IOBUS_TIMER1         = 66,
CLOCK_IOBUS_TIMER2         = 67,
CLOCK_IOBUS_TIMER3         = 68,
CLOCK_IOBUS_TIMER4         = 69,
CLOCK_IOBUS_TIMER5         = 70,
CLOCK_IOBUS_WDT            = 71,
CLOCK_IOBUS_AXI_BRIGE_M    = 72,
CLOCK_IOBUS_AXI_BRIGE_S    = 73,
CLOCK_IOBUS_GPSB_GRP       = 74,
CLOCK_IOBUS_GPSB0          = 75,
CLOCK_IOBUS_GPSB1          = 76,
CLOCK_IOBUS_GPSB2          = 77,
CLOCK_IOBUS_GPSB3          = 78,
CLOCK_IOBUS_GPSB4          = 79,
CLOCK_IOBUS_GPSB5          = 80,
CLOCK_IOBUS_GPSB_BUS       = 81,
CLOCK_IOBUS_GPSB_M         = 82,
CLOCK_IOBUS_CMU            = 83,
CLOCK_IOBUS_SYSSM          = 84,
CLOCK_IOBUS_MAX            = 85
} CLOCKIobus_t;

```

Description

Value	Description
<i>CLOCK_IOBUS_XXXX</i>	<i>Peripheral io bus clock control offset. Refer to MICOM Sub-system Configuration Register Map (0x1b930000) in “TCC805x Full Specification” [1].</i>
<i>CLOCK_IOBUS_MAX</i>	<i>Maximum Peripheral clock control offset</i>

Remark

None

5 CONSTANTS

5.1 Clock Rate

Constant value for the pre-defined reference clock rate

Declaration

```
#define CLOCK_XIN_CLK_RATE    ( 24000000 )    // 24MHz
#define CLOCK_XTIN_CLK_RATE  32768            // 32.768kHz
```

Description

<i>Value</i>	<i>Description</i>
<i>CLOCK_XIN_CLK_RATE</i>	<i>Clock signal from PMU is 24 MHz.</i>
<i>CLOCK_XTIN_CLK_RATE</i>	<i>Clock signal from RTC is 32.768 kHz.</i>

Remark

None

6 STRUCTURES

6.1 CLOCKPms_t

Structure for PLL rate control

Declaration

```
typedef struct CLOCK_PMS
{
    uint32    uiFpLL;
    uint32    uiEn;
    uint32    uiP;
    uint32    uiM;
    uint32    uiS;
    uint32    uiSrc;
} CLOCKPms_t;
```

Description

Value	Description
<i>uiFpLL</i>	<i>PLL value</i>
<i>uiEn</i>	<i>RESETB: PLL Reset when Active Low. 0: PLL stops and all digital blocks are reset. 1: PLL starts its normal operation after lock time. (refer to 0x1B931000 in “TCC805x Full Specification” [1])</i>
<i>uiP</i>	<i>P: Value of the 6-bit programmable pre-divider. (refer to 0x1B931000 in “TCC805x Full Specification” [1])</i>
<i>uiM</i>	<i>M: Value of the 10-bit programmable main-divider (refer to 0x1B931000 in “TCC805x Full Specification” [1])</i>
<i>uiS</i>	<i>S: Value of the 3-bit programmable Scaler (refer to 0x1B931000 in “TCC805x Full Specification” [1])</i>
<i>uiSrc</i>	<i>FIN_SEL: FIN source clock selection (refer to 0x1B931000 in “TCC805x Full Specification” [1])</i>

Remark

None

6.2 CLOCKClkCtrl_t

Structure for PLL clock control register

Declaration

```
typedef struct CLOCK_CLK_CTRL
{
    uint32    uiFreq;
    uint32    uiEn;
    uint32    uiConf;
    uint32    uiSel;
} CLOCKClkCtrl_t;
```

Description

Value	Description
<i>uiFreq</i>	<i>Clock rate</i>
<i>uiEn</i>	<i>Enable</i>
<i>uiConf</i>	<i>XIN divider's divisor</i>
<i>uiSel</i>	<i>Source clock selection</i>

Remark

None

6.3 CLOCKPclkCtrl_t

Structure for peripheral PLL clock control register

Declaration

```
typedef struct CLOCK_PCLK_CTRL
{
    uint32_t    uiPeriName;
    uint32_t    uiFreq;
    uint32_t    uiMd;
    uint32_t    uiEn;
    uint32_t    uiSel;
    uint32_t    uiDivVal;
} CLOCKPclkCtrl_t;
```

Description

Value	Description
uiPeriName	Peripheral name
uiFreq	Clock rate
uiMd	Mode divider
uiEn	Enable
uiSel	Source clock
uiDivVal	Divisor value

Remark

None

7 APIs

7.1 CLOCK_Init

Initializes the clock controller.

Declaration

```
void CLOCK_Init  
(  
    void  
);
```

Parameters

None

Return

None

Remark

None

7.2 CLOCK_SetPIIRate

Controls the *PLL0PMS* and *PLL1PMS* register. (Refer to *0x1B931000* in “*TCC805x Full Specification*”. [1])

Declaration

```
sint32 CLOCK_SetPLLRate
(
    sint32    iId,
    uint32    uiRate
);
```

Parameters

Value	Description
iId	MPLL index
uiRate	Clock rate

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.3 CLOCK_GetPllRate

Gets the PLL rate.

Declaration

```
uint32 CLOCK_GetPllRate  
(  
    sint32    iId  
);
```

Parameters

<i>Value</i>	<i>Description</i>
<i>iId</i>	<i>Clock index</i>

Return

<i>Value</i>	<i>Description</i>
<i>uint32</i>	<i>Clock rate</i>

Remark

None

7.4 CLOCK_SetPLLDiv

Sets the PLL divisor value.

Declaration

```
sint32 CLOCK_SetPLLDiv  
(  
    sint32    iId,  
    uint32    uiPLldiv  
);
```

Parameters

Value	Description
<i>iId</i>	<i>Clock index</i>
<i>uiPLldiv</i>	<i>PLL divisor value</i>

Return

Value	Description
<i>sint32</i>	<i>0: Success -1: Failure</i>

Remark

None

7.5 CLOCK_SetClkCtrlRate

Controls the *CPU_CLK_CTRL* register.

Declaration

```
sint32 CLOCK_SetClkCtrlRate
(
    sint32    iId,
    uint32    uiRate
);
```

Parameters

Value	Description
iId	Clock index
uiRate	Clock rate

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.6 CLOCK_GetClkCtrlRate

Gets the clock rate for *CPU_CLK_CTRL* register.

Declaration

```
uint32 CLOCK_GetClkCtrlRate
(
    sint32    iId
);
```

Parameters

Value	Description
iId	Clock index

Return

Value	Description
uint32	Clock rate

Remark

None

7.7 CLOCK_IsPeriEnabled

Checks the peripheral enable status.

Declaration

```
sint32 CLOCK_IsPeriEnabled  
(  
    sint32    iId  
);
```

Parameters

Value	Description
<i>iId</i>	<i>Clock index</i>

Return

Value	Description
<i>sint32</i>	<i>0: Disabled</i> <i>1: Enabled</i>

Remark

None

7.8 CLOCK_EnablePeri

Enables the peripheral clock.

Declaration

```
sint32 CLOCK_EnablePeri
(
    sint32    iId
);
```

Parameters

Value	Description
iId	Clock index

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.9 CLOCK_DisablePeri

Disables the peripheral clock.

Declaration

```
sint32 CLOCK_DisablePeri  
(  
    sint32    iId  
);
```

Parameters

<i>Value</i>	<i>Description</i>
<i>iId</i>	<i>Clock index</i>

Return

<i>Value</i>	<i>Description</i>
<i>sint32</i>	<i>0: Success</i> <i>-1: Failure</i>

Remark

None

7.10 CLOCK_GetPeriRate

Gets the peripheral clock rate.

Declaration

```
uint32 CLOCK_GetPeriRate
(
    sint32    iId
);
```

Parameters

<i>Value</i>	<i>Description</i>
<i>iId</i>	<i>Clock index</i>

Return

<i>Value</i>	<i>Description</i>
<i>uint32</i>	<i>Peri clock rate</i>

Remark

None

7.11 CLOCK_SetPeriRate

Sets the peripheral clock rate (*CLOCK_MCKC_PCLKCTRL* register).

Declaration

```
sint32 CLOCK_SetPeriRate
(
    sint32    iId,
    uint32    uiRate
);
```

Parameters

Value	Description
iId	Clock index
uiRate	Clock rate

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.12 CLOCK_IsIobusPwn

Gets the active state of the peripheral bus clock mask status (*HCLK_MASK#* register).

Declaration

```
sint32 CLOCK_IsIobusPwn  
(  
    sint32    iId,  
);
```

Parameters

Value	Description
<i>iId</i>	<i>Bus clock index</i>

Return

Value	Description
<i>sint32</i>	<i>0: Enabled bus clock 1: Disabled bus clock -1: Wrong first parameter</i>

Remark

None

7.13 CLOCK_EnableIobus

Enables peripheral bus clock (*HCLK_MASK#*, *SW_RESET#* register).

Declaration

```
sint32 CLOCK_EnableIobus
(
    sint32    iId,
    boolean    bEn
);
```

Parameters

Value	Description
iId	Bus clock index
bEn	Bus clock enable

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.14 CLOCK_SetIobusPwn

Disable the peripheral bus clock mask status (*HCLK_MASK#* register).

Declaration

```
sint32 CLOCK_SetIobusPwn
(
    sint32    iId,
    boolean    bEn
);
```

Parameters

Value	Description
iId	Bus clock index
bEn	1: Bus clock is disabled

Return

Value	Description
sint32	0: Success -1: Failure

Remark

None

7.15 CLOCK_SetSwReset

Controls the *SW_RESET#* register.

Declaration

```
sint32 CLOCK_SetSwReset  
(  
    sint32    iId,  
    boolean   bReset  
);
```

Parameters

Value	Description
<i>iId</i>	<i>Bus clock index</i>
<i>bReset</i>	<i>Enable to reset bus clock.</i>

Return

Value	Description
<i>sint32</i>	<i>0: Success -1: Failure</i>

Remark

None

8 HOW TO USE CLOCK DRIVER

The clock driver must be initialized first in the BSP initialization process. And each driver that needs a clock must set the peripheral clock during its own initialization.

8.1 Initialize CLOCK Driver

```
{  
    CLOCK_Init();  
    ...  
}
```

8.2 Set Peripheral Clock

```
{  
    ...  
    uint32 rate = 48000000UL;  
  
    /* Enable/Disable the xxx controller peripheral clock */  
    regRet = CLOCK_SetPeriRate(CLOCK_PERI_xxx, rate);  
    regRet = CLOCK_EnablePeri(CLOCK_PERI_xxx);  
    rate = CLOCK_GetPeriRate(CLOCK_PERI_xxx);  
    printf("rate is x%x \n", rate);  
    ...  
    regRet = CLOCK_DisablePeri(CLOCK_PERI_xxx);  
    ...  
}
```

8.3 Set Peripheral Bus Clock

```
{  
    ...  
    /* Enable/Disable the xxx controller peripheral bus clock*/  
    regRet = CLOCK_EnableIobus(CLOCK_IOBUS_xxx, SALEnabled);  
    regRet = CLOCK_EnableIobus(CLOCK_IOBUS_xxx, SALDisabled);  
}
```

9 REFERENCES

- [1] TCC805x Full Specification
- [2] Contact Telechips for more details: sales@telechips.com

10 REVISION HISTORY

Rev. 1.01: 2021-01-22

- Updated
 - Chapter 5.1: Declaration
 - Chapter 6.1: Description
 - Chapter 7.7, Chapter 7.10: Mismatched variable name and description
 - Chapter 7.12, Chapter 7.13, Chapter 7.14: Declaration and parameter

Rev. 1.00: 2020-11-12

- First version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."

"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."